

Week-01 HandsOn Solution

❖ Design Patterns and Principles

Exercise 1: Implementing the Singleton Pattern

- **Code:**

```
using System;

public class Logger {
    private static Logger? instance;
    private static readonly object lockObj = new object();

    private Logger() { }

    public static Logger Instance
    {
        get
        {
            lock (lockObj)
            {
                if (instance == null)
                    instance = new Logger();
                return instance;
            }
        }
    }

    public void Log(string message) {
        Console.WriteLine("Log: " + message);
    }
}

public class Program {
    public static void Main()
    {
        Logger log1 = Logger.Instance;
        Logger log2 = Logger.Instance;
        log1.Log("First message");
        Console.WriteLine(log1 == log2);
    }
}
```

- **Output:**

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
C:\Users\KIIT\Desktop\Exe_Week1_Engineering concepts_code\Design Patterns and Principles\Q1>dotnet run
Log: First message
True
```

Exercise 2: Implementing the Factory Method Pattern

- **Code:**

```
using System;

interface IDocument {
    void Open();
}

class WordDocument : IDocument {
    public void Open() => Console.WriteLine("Word document opened.");
}

class PdfDocument : IDocument {
    public void Open() => Console.WriteLine("PDF document opened.");
}

abstract class DocumentFactory {
    public abstract IDocument CreateDocument();
}

class WordFactory : DocumentFactory {
    public override IDocument CreateDocument() => new WordDocument();
}

class PdfFactory : DocumentFactory {
    public override IDocument CreateDocument() => new PdfDocument();
}

public class Program {
    public static void Main() {
        DocumentFactory factory = new PdfFactory();
        IDocument doc = factory.CreateDocument();
        doc.Open();
    }
}
```

- **Output:**

```
C:\Users\KIIT\Desktop\Exe_Week1_Engineering concepts_code\Design Patterns and Principles\Q2>dotnet run
PDF document opened.
C:\Users\KIIT\Desktop\Exe_Week1_Engineering concepts_code\Design Patterns and Principles\Q2>
```

Exercise 3: Implementing the Builder Pattern

- **Code:**

```
using System;

class Computer {
    public string CPU { get; private set; }
    public string RAM { get; private set; }
    public string Storage { get; private set; }

    private Computer() { }

    public class Builder {
        private Computer computer = new Computer();

        public Builder SetCPU(string cpu) { computer.CPU = cpu; return this; }
        public Builder SetRAM(string ram) { computer.RAM = ram; return this; }
        public Builder SetStorage(string storage) { computer.Storage = storage; return this; }
        public Computer Build() => computer;
    }
}

public class Program {
    public static void Main() {
        var pc = new Computer.Builder().SetCPU("i7").SetRAM("16GB").SetStorage("1TB").Build();
    }
}
```

```

        Console.WriteLine($"CPU: {pc.CPU}, RAM: {pc.RAM}, Storage: {pc.Storage}");
    }
}

```

- **Output:**

```

C:\Users\KIIT\Desktop\Exe_Week1_Engineering concepts_code\Design Patterns and Principles\Q3>dotnet run
CPU: i7, RAM: 16GB, Storage: 1TB

C:\Users\KIIT\Desktop\Exe_Week1_Engineering concepts_code\Design Patterns and Principles\Q3>

```

Exercise 4: Implementing the Adapter Pattern

- **Code:**

```

using System;

interface IPaymentProcessor{
    void ProcessPayment();
}

class OldPaymentGateway{
    public void MakePayment() => Console.WriteLine("Payment done using old gateway.");
}

class PaymentAdapter : IPaymentProcessor{
    private OldPaymentGateway gateway = new OldPaymentGateway();
    public void ProcessPayment() => gateway.MakePayment();
}

public class Program{
    public static void Main(){
        IPaymentProcessor processor = new PaymentAdapter();
        processor.ProcessPayment();
    }
}

```

- **Output:**

```

C:\Users\KIIT\Desktop\Exe_Week1_Engineering concepts_code\Design Patterns and Principles\Q4>dotnet run
Payment done using old gateway.

C:\Users\KIIT\Desktop\Exe_Week1_Engineering concepts_code\Design Patterns and Principles\Q4>

```

Exercise 5: Implementing the Decorator Pattern

- **Code:**

```

using System;

interface INotifier{
    void Send(string message);
}

class EmailNotifier : INotifier{
    public void Send(string message) => Console.WriteLine("Email: " + message);
}

```

```

abstract class NotifierDecorator : INotifier{
    protected INotifier wrappee;
    public NotifierDecorator(INotifier notifier) { wrappee=notifier; }
    public virtual void Send(string message) => wrappee.Send(message);
}

class SMSNotifier : NotifierDecorator{
    public SMSNotifier(INotifier notifier) : base(notifier) { }
    public override void Send(string message)
    {
        base.Send(message);
        Console.WriteLine("SMS: "+message);
    }
}

public class Program{
    public static void Main(){
        INotifier notifier=new SMSNotifier(new EmailNotifier());
        notifier.Send("Hello!");
    }
}

```

- **Output:**

```

C:\Users\KIIT\Desktop\Exe_Week1_Engineering concepts_code\Design Patterns and Principles\Q5>dotnet run
Email: Hello!
SMS: Hello!

```

Exercise 6: Implementing the Proxy Pattern

- **Code:**

```

using System;

interface IImage{
    void Display();
}

class RealImage : IImage{
    private string filename;

    public RealImage(string filename){
        this.filename=filename;
        LoadFromDisk();
    }
    private void LoadFromDisk(){
        Console.WriteLine("Loading "+filename);
    }

    public void Display(){
        Console.WriteLine("Displaying "+filename);
    }
}

class ProxyImage : IImage{
    private RealImage? realImage;
    private string filename;

    public ProxyImage(string filename){
        this.filename=filename;
    }

    public void Display(){
        if (realImage==null)
        {
            realImage=new RealImage(filename);
        }
        realImage.Display();
    }
}

```

```

class Program{
    static void Main(){
        Image img = new ProxyImage("image1.jpg");
        img.Display();
        img.Display();
    }
}

```

- **Output:**

```

C:\Users\KIIT\Desktop\Exe_Week1_Engineering concepts_code\Design Patterns and Principles\Q6>dotnet run
Loading image1.jpg
Displaying image1.jpg
Displaying image1.jpg

```

Exercise 7: Implementing the Observer Pattern

- **Code:**

```

using System;
using System.Collections.Generic;

interface IObservable{
    void Update(float price);
}

interface IStock{
    void Register(IObservable observer);
    void Unregister(IObservable observer);
    void NotifyObservers();
}

class StockMarket : IStock{
    private List<IObservable> observers = new();
    private float price;

    public void Register(IObservable observer) => observers.Add(observer);
    public void Unregister(IObservable observer) => observers.Remove(observer);
    public void SetPrice(float price){
        this.price = price;
        NotifyObservers();
    }

    public void NotifyObservers(){
        foreach (var o in observers)
        {
            o.Update(price);
        }
    }
}

class MobileApp : IObservable{
    public void Update(float price)
    {
        Console.WriteLine("Mobile App: Stock price updated to "+price);
    }
}

class Program{
    static void Main(){
        var stock = new StockMarket();
        var mobile = new MobileApp();
        stock.Register(mobile);
        stock.SetPrice(99.99f);
    }
}

```

- **Output:**

```
C:\Users\KIIT\Desktop\Exe_Week1_Engineering concepts_code\Design Patterns and Principles\Q7>dotnet run
Mobile App: Stock price updated to 99.99

C:\Users\KIIT\Desktop\Exe_Week1_Engineering concepts_code\Design Patterns and Principles\Q7>
```

Exercise 8: Implementing the Strategy Pattern

- **Code:**

```
using System;

interface IPaymentStrategy{
    void Pay();
}

class CreditCardPayment : IPaymentStrategy{
    public void Pay() => Console.WriteLine("Paid using Credit Card");
}

class PayPalPayment : IPaymentStrategy{
    public void Pay() => Console.WriteLine("Paid using PayPal");
}

class PaymentContext{
    private IPaymentStrategy strategy;

    public PaymentContext(IPaymentStrategy strategy){
        this.strategy = strategy;
    }

    public void Pay(){
        strategy.Pay();
    }
}

class Program{
    static void Main(){
        var context = new PaymentContext(new PayPalPayment());
        context.Pay();
    }
}
```

- **Output:**

```
C:\Users\KIIT\Desktop\Exe_Week1_Engineering concepts_code\Design Patterns and Principles\Q8>dotnet run
Paid using PayPal

C:\Users\KIIT\Desktop\Exe_Week1_Engineering concepts_code\Design Patterns and Principles\Q8>
```

Exercise 9: Implementing the Command Pattern

- **Code:**

```
using System;

interface ICommand{
    void Execute();
}

class Light{
    public void On() => Console.WriteLine("Light is ON");
    public void Off() => Console.WriteLine("Light is OFF");
}

class LightOnCommand : ICommand{
    private Light light;
```

```

        public LightOnCommand(Light light) { this.light = light; }
        public void Execute() => light.On();
    }

    class LightOffCommand : ICommand {
        private Light light;
        public LightOffCommand(Light light) { this.light = light; }
        public void Execute() => light.Off();
    }

    class RemoteControl {
        private ICommand command;
        public void SetCommand(ICommand cmd) => command = cmd;
        public void PressButton() => command.Execute();
    }

    class Program {
        static void Main() {
            var light = new Light();
            var remote = new RemoteControl();

            remote.SetCommand(new LightOnCommand(light));
            remote.PressButton();

            remote.SetCommand(new LightOffCommand(light));
            remote.PressButton();
        }
    }

```

• Output:

```

C:\Users\KIIT\Desktop\Exe_Week1_Engineering concepts_code\Design Patterns and Principles\Q9>dotnet run
Light is ON
Light is OFF

```

Exercise 10: Implementing the MVC Pattern

• Code:

```

using System;

class Student {
    public string Name, Id, Grade;
    public Student(string name, string id, string grade) => (Name, Id, Grade) = (name, id, grade);
}

class StudentView {
    public void Show(string name, string id, string grade) =>
        Console.WriteLine($"Student Details:\nName: {name}\nID: {id}\nGrade: {grade}\n");
}

class StudentController {
    Students; StudentView v;
    public StudentController(Students, StudentView v) => (this.s, this.v) = (s, v);
    public void SetGrade(string g) => s.Grade = g;
    public void Display() => v.Show(s.Name, s.Id, s.Grade);
}

class Program {
    static void Main() {
        var student = new Student("Gaurav", "22052116", "A");
        var view = new StudentView();
        var controller = new StudentController(student, view);

        controller.Display();
        Console.WriteLine("Updating Grade from A to A+...\n");
        controller.SetGrade("A+");
        controller.Display();
    }
}

```

- **Output:**

```
C:\Users\KIIT\Desktop\Exe_Week1_Engineering concepts_code\Design Patterns and Principles\Q10>dotnet run
Student Details:
Name: Gaurav
ID: 22052116
Grade: A

Updating Grade from A to A+...

Student Details:
Name: Gaurav
ID: 22052116
Grade: A+
```

Exercise 11: Implementing Dependency Injection

- **Code:**

```
using System;

interface ICustomerRepository { string FindCustomerById(int id); }

class CustomerRepositoryImpl : ICustomerRepository {
    public string FindCustomerById(int id) {
        if (id == 22052116) return $"ID: {id}, Name: Gaurav Kumar";
        return $"Customer #{id} not found";
    }
}

class CustomerService {
    ICustomerRepository repo;
    public CustomerService(ICustomerRepository repo) => this.repo = repo;
    public void PrintCustomer(int id) => Console.WriteLine(repo.FindCustomerById(id));
}

class Program {
    static void Main() {
        var service = new CustomerService(new CustomerRepositoryImpl());
        service.PrintCustomer(22052116);
    }
}
```

- **Output:**

```
C:\Users\KIIT\Desktop\Exe_Week1_Engineering concepts_code\Design Patterns and Principles\Q11>dotnet run
ID: 22052116, Name: Gaurav Kumar

C:\Users\KIIT\Desktop\Exe_Week1_Engineering concepts_code\Design Patterns and Principles\Q11>
```


❖ Algorithms and Data Structures

Exercise 1: Inventory Management System

- Code:

```
using System;
using System.Collections.Generic;

class Product {
    public int ProductId { get; set; }
    public string ProductName { get; set; }
    public int Quantity { get; set; }
    public double Price { get; set; }

    public Product(int productId, string productName, int quantity, double price) {
        ProductId = productId;
        ProductName = productName;
        Quantity = quantity;
        Price = price;
    }

    public override string ToString() {
        return $"ID: {ProductId}, Name: {ProductName}, Qty: {Quantity}, Price: {Price}";
    }
}

class InventoryManager {
    private Dictionary<int, Product> inventory = new Dictionary<int, Product>();

    public void AddProduct(Product product) {
        inventory[product.ProductId] = product;
    }

    public void UpdateProduct(int productId, int newQty, double newPrice) {
        if (inventory.ContainsKey(productId)) {
            inventory[productId].Quantity = newQty;
            inventory[productId].Price = newPrice;
        }
    }

    public void DeleteProduct(int productId) {
        inventory.Remove(productId);
    }

    public void DisplayInventory() {
        foreach (var item in inventory.Values) {
            Console.WriteLine(item);
        }
    }
}

class Program {
    static void Main(string[] args) {
        InventoryManager manager = new InventoryManager();

        manager.AddProduct(new Product(1, "Keyboard", 10, 500.0));
        manager.AddProduct(new Product(2, "Mouse", 15, 300.0));
        manager.DisplayInventory();

        Console.WriteLine("\nUpdating product 2...");
        manager.UpdateProduct(2, 20, 320.0);

        Console.WriteLine("\nDeleting product 1...");
        manager.DeleteProduct(1);

        Console.WriteLine("\nFinal Inventory:");
        manager.DisplayInventory();
    }
}
```

- **Output:**

```
C:\Users\KIIT\Desktop\Exe_Week1_Engineering_concepts_code\Algorithms_Data Structures\Q1>dotnet run
ID: 1, Name: Keyboard, Qty: 10, Price: 500
ID: 2, Name: Mouse, Qty: 15, Price: 300

Updating product 2...

Deleting product 1...

Final Inventory:
ID: 2, Name: Mouse, Qty: 20, Price: 320
```

- **Analysis:**

Using Dictionary<int, Product> (i.e., HashMap):

- Add, Update, Delete - O(1) average time.
- Optimized for large datasets via constant-time hashing.

Exercise 2: E-commerce Platform Search Function

- **Code:**

```
using System;

class Product {
    public int ProductId { get; set; }
    public string ProductName { get; set; }
    public string Category { get; set; }

    public Product(int productId, string productName, string category) {
        ProductId = productId;
        ProductName = productName;
        Category = category;
    }

    public override string ToString() {
        return $"ID: {ProductId}, Name: {ProductName}, Category: {Category}";
    }
}

class SearchFunction {
    public static int LinearSearch(Product[] products, string name) {
        for (int i = 0; i < products.Length; i++) {
            if (products[i].ProductName.Equals(name, StringComparison.OrdinalIgnoreCase)) {
                return i;
            }
        }
        return -1;
    }

    public static int BinarySearch(Product[] products, string name) {
        int left = 0, right = products.Length - 1;
        while (left <= right) {
            int mid = left + (right - left) / 2;
            int cmp = string.Compare(products[mid].ProductName, name, StringComparison.OrdinalIgnoreCase);
            if (cmp == 0)
                return mid;
            elseif (cmp < 0)
                left = mid + 1;
            else
                right = mid - 1;
        }
        return -1;
    }
}
```

```

class Program {
    static void Main(string[] args) {
        Product[] products = {
            new Product(1, "Apple", "Fruits"),
            new Product(2, "Banana", "Fruits"),
            new Product(3, "Carrot", "Vegetables"),
            new Product(4, "Mango", "Fruits")
        };

        Console.WriteLine("=== Linear Search ===");
        int index = SearchFunction.LinearSearch(products, "Carrot");
        Console.WriteLine(index != -1 ? $"Found: {products[index]}": "Product not found");

        Array.Sort(products, (p1, p2) => p1.ProductName.CompareTo(p2.ProductName));

        Console.WriteLine("\n=== Binary Search ===");
        index = SearchFunction.BinarySearch(products, "Mango");
        Console.WriteLine(index != -1 ? $"Found: {products[index]}": "Product not found");
    }
}

```

• Output:

```

C:\Users\KIIT\Desktop\Exe_Week1_Engineering_concepts_code\Algorithms_Data Structures\Q2>dotnet run
=== Linear Search ===
Found: ID: 3, Name: Carrot, Category: Vegetables

=== Binary Search ===
Found: ID: 4, Name: Mango, Category: Fruits

```

• Analysis:

- Linear Search: **O(n)**
- Binary Search: **O(log n)** (on sorted data)
- Use Binary Search for performance-critical scenarios with sorted data.

Exercise 3: Sorting Customer Orders

• Code:

```

using System;

class Order {
    public int OrderId { get; set; }
    public string CustomerName { get; set; }
    public double TotalPrice { get; set; }

    public Order(int id, string name, double price) {
        OrderId = id;
        CustomerName = name;
        TotalPrice = price;
    }

    public override string ToString() {
        return $"OrderID: {OrderId}, Name: {CustomerName}, Price: {TotalPrice}";
    }
}

class Program {
    static void BubbleSort(Order[] orders) {

```

```

        intn=orders.Length;
        for (inti=0; i<n-1; i++){
            for (intj=0; j<n-i-1; j++){
                if (orders[j].TotalPrice>orders[j+1].TotalPrice){
                    vartemp=orders[j];
                    orders[j] =orders[j+1];
                    orders[j+1] =temp;
                }
            }
        }
    }

    staticvoidQuickSort(Order[] orders, intlow, inthigh){
        if (low<high){
            intpi=Partition(orders, low, high);
            QuickSort(orders, low, pi-1);
            QuickSort(orders, pi+1, high);
        }
    }

    staticintPartition(Order[] orders, intlow, inthigh){
        doublepivot=orders[high].TotalPrice;
        inti=low-1;
        for (intj=low; j<high; j++){
            if (orders[j].TotalPrice<pivot){
                i++;
                vartemp=orders[i];
                orders[i] =orders[j];
                orders[j] =temp;
            }
        }
        vartemp2=orders[i+1];
        orders[i+1] =orders[high];
        orders[high] =temp2;
        returni+1;
    }

    staticvoidPrintOrders(Order[] orders){
        foreach (varorderinorders){
            Console.WriteLine(order);
        }
    }

    staticvoidMain(string[] args){
        Order[] orders= {
            newOrder(101, "Alice", 5000),
            newOrder(102, "Bob", 2500),
            newOrder(103, "Charlie", 8000)
        };

        Console.WriteLine("Bubble Sort:");
        varbubbleSorted= (Order[])orders.Clone();
        BubbleSort(bubbleSorted);
        PrintOrders(bubbleSorted);

        Console.WriteLine("\nQuick Sort:");
        varquickSorted= (Order[])orders.Clone();
        QuickSort(quickSorted, 0, quickSorted.Length-1);
        PrintOrders(quickSorted);
    }
}

```

• Output:

```

C:\Users\KIIT\Desktop\Exe_Week1_Engineering concepts_code\Algorithms_Data Structures\Q3>dotnet run
Bubble Sort:
OrderID: 102, Name: Bob, Price: 2500
OrderID: 101, Name: Alice, Price: 5000
OrderID: 103, Name: Charlie, Price: 8000

Quick Sort:
OrderID: 102, Name: Bob, Price: 2500
OrderID: 101, Name: Alice, Price: 5000
OrderID: 103, Name: Charlie, Price: 8000

```

- Analysis:

- Bubble Sort: $O(n^2)$
- Quick Sort: $O(n \log n)$ average case
- Quick Sort is better for larger datasets due to divide-and-conquer strategy.

Exercise 4: Employee Management System

- Code:

```
using System;

class Employee {
    public int EmployeeId;
    public string Name;
    public string Position;
    public double Salary;

    public Employee(int id, string name, string pos, double sal) {
        EmployeeId = id;
        Name = name;
        Position = pos;
        Salary = sal;
    }

    public void Display() {
        Console.WriteLine($"ID: {EmployeeId}, Name: {Name}, Position: {Position}, Salary: {Salary}");
    }
}

class Program {
    static void Main(string[] args) {
        Employee[] employees = new Employee[100];
        int count = 0;

        employees[count++] = new Employee(1, "Alice", "Manager", 60000);
        employees[count++] = new Employee(2, "Bob", "Engineer", 45000);

        Console.WriteLine("All Employees:");
        for (int i = 0; i < count; i++) {
            employees[i].Display();
        }

        Console.WriteLine("\nSearching Employee with ID 2:");
        for (int i = 0; i < count; i++) {
            if (employees[i].EmployeeId == 2) {
                employees[i].Display();
                break;
            }
        }

        Console.WriteLine("\nDeleting Employee with ID 1:");
        for (int i = 0; i < count; i++) {
            if (employees[i].EmployeeId == 1) {
                for (int j = i; j < count - 1; j++) {
                    employees[j] = employees[j + 1];
                }
                count--;
                break;
            }
        }

        Console.WriteLine("\nEmployees after deletion:");
        for (int i = 0; i < count; i++) {
            employees[i].Display();
        }
    }
}
```

- **Output:**

```
C:\Users\KIIT\Desktop\Exe_Week1_Engineering_concepts_code\Algorithms_Data Structures\Q4>dotnet run
All Employees:
ID: 1, Name: Alice, Position: Manager, Salary: 60000
ID: 2, Name: Bob, Position: Engineer, Salary: 45000

Searching Employee with ID 2:
ID: 2, Name: Bob, Position: Engineer, Salary: 45000

Deleting Employee with ID 1:

Employees after deletion:
ID: 2, Name: Bob, Position: Engineer, Salary: 45000
```

- **Analysis:**

- Add/Search/Delete: **O(n)** worst-case
- Arrays are fast for indexed access, but not efficient for dynamic data or frequent insertions/deletions.

Exercise 5: Task Management System

- **Code:**

```
using System;

class Task {
    public int TaskId;
    public string TaskName;
    public string Status;
    public Task? Next;

    public Task(int id, string name, string status) {
        TaskId = id;
        TaskName = name;
        Status = status;
        Next = null;
    }

    public void Display() =>
        Console.WriteLine($"ID: {TaskId}, Name: {TaskName}, Status: {Status}");
}

class TaskManager {
    private Task? head;

    public void AddTask(int id, string name, string status) {
        var newTask = new Task(id, name, status);
        if (head == null) head = newTask;
        else {
            var temp = head;
            while (temp.Next != null) temp = temp.Next;
            temp.Next = newTask;
        }
    }

    public void DisplayTasks() {
        var temp = head;
        while (temp != null) {
            temp.Display();
            temp = temp.Next;
        }
    }

    public void DeleteTask(int id) {
```

```

        Task? temp=head, prev=null;

        if (temp!=null&&temp.TaskId==id){
            head=temp.Next;
            return;
        }

        while (temp!=null&&temp.TaskId!=id){
            prev=temp;
            temp=temp.Next;
        }

        if (temp==null||prev==null) return;

        prev.Next=temp.Next;
    }
}

class Program
{
    static void Main(){
        var manager=new TaskManager();
        manager.AddTask(1, "Design", "Pending");
        manager.AddTask(2, "Code", "In Progress");
        manager.AddTask(3, "Test", "Pending");

        Console.WriteLine("All Tasks:");
        manager.DisplayTasks();

        Console.WriteLine("\nDeleting Task with ID 2...");
        manager.DeleteTask(2);
        manager.DisplayTasks();
    }
}

```

• Output:

```

C:\Users\KIIT\Desktop\Exe_Week1_Engineering_concepts_code\Algorithms_Data Structures\Q5>dotnet run
All Tasks:
ID: 1, Name: Design, Status: Pending
ID: 2, Name: Code, Status: In Progress
ID: 3, Name: Test, Status: Pending

Deleting Task with ID 2...
ID: 1, Name: Design, Status: Pending
ID: 3, Name: Test, Status: Pending

```

• Analysis:

- Add: **O(n)** (if added at end)
- Delete/Search: **O(n)**
- Linked Lists are better for dynamic size and frequent insertions/deletions without shifting elements.

Exercise 6: Library Management System

• Code:

```

using System;

class Book {
    public int BookId;
    public string Title;
    public string Author;

    public Book(int id, string title, string author) {

```

```

        BookId=id;
        Title=title;
        Author=author;
    }

    public void Display() {
        Console.WriteLine($"ID: {BookId}, Title: {Title}, Author: {Author}");
    }
}

class Program {
    static int LinearSearch(Book[] books, string title) {
        for (int i=0; i<books.Length; i++) {
            if (books[i].Title.Equals(title, StringComparison.OrdinalIgnoreCase)) return i;
        }
        return -1;
    }

    static int BinarySearch(Book[] books, string title) {
        int low=0, high=books.Length-1;
        while (low<=high) {
            int mid= (low+high) /2;
            int cmp=string.Compare(books[mid].Title, title, StringComparison.OrdinalIgnoreCase);
            if (cmp==0) return mid;
            elseif (cmp<0) low=mid+1;
            elseif (cmp>0) high=mid-1;
        }
        return -1;
    }

    static void Main(string[] args) {
        Book[] books= {
            new Book(1, "Algorithms", "Cormen"),
            new Book(2, "Data Structures", "Lafore"),
            new Book(3, "Clean Code", "Martin")
        };

        Console.WriteLine("Linear Search for 'Data Structures':");
        int index=LinearSearch(books, "Data Structures");
        if (index!=-1) books[index].Display();

        Array.Sort(books, (b1, b2) => b1.Title.CompareTo(b2.Title));
        Console.WriteLine("\nBinary Search for 'Clean Code':");
        index=BinarySearch(books, "Clean Code");
        if (index!=-1) books[index].Display();
    }
}

```

• Output:

```

C:\Users\KIIT\Desktop\Exe_Week1_Engineering_concepts_code\Algorithms_Data Structures\Q6>dotnet run
Linear Search for 'Data Structures':
ID: 2, Title: Data Structures, Author: Lafore

Binary Search for 'Clean Code':
ID: 3, Title: Clean Code, Author: Martin

```

• Analysis:

- Linear: **O(n)** – good for unsorted or small data sets.
- Binary: **O(log n)** – works only on sorted arrays.
- Binary Search is better for frequent search operations with large sorted data.

Exercise 7: Financial Forecasting

- **Code:**

```
using System;

class Program {
    static double PredictFutureValue(double currentValue, double growthRate, int years) {
        if (years == 0) return currentValue;
        return PredictFutureValue(currentValue * (1 + growthRate), growthRate, years - 1);
    }

    static double PredictUsingLoop(double currentValue, double growthRate, int years) {
        for (int i = 0; i < years; i++) {
            currentValue *= (1 + growthRate);
        }
        return currentValue;
    }

    static void Main(string[] args) {
        double initial = 10000;
        double rate = 0.1;
        int years = 5;

        double futureRecursive = PredictFutureValue(initial, rate, years);
        double futureIterative = PredictUsingLoop(initial, rate, years);

        Console.WriteLine($"Future Value (Recursive): {futureRecursive}");
        Console.WriteLine($"Future Value (Iterative): {futureIterative}");
    }
}
```

- **Output:**

```
C:\Users\KIIT\Desktop\Exe_Week1_Engineering_concepts_code\Algorithms_Data Structures\Q7>dotnet run
Future Value (Recursive): 16105.100000000008
Future Value (Iterative): 16105.100000000008
```

- **Analysis:**

- Recursive: **$O(n)$** time, **$O(n)$** space (stack)
- Optimized (iterative): **$O(n)$** time, **$O(1)$** space
- Use loops or memoization to reduce stack overhead.

Week-02 HandsOn Solution

❖ Advanced SQL Server

File 1: Advanced Concepts

Exercise-1: Ranking and Window Functions

- Code:

//Database Creation:

```
CREATEDATABASEOnlineRetailStore;
GO

USE OnlineRetailStore;
GO

CREATETABLEProducts (
    ProductID INTIDENTITY(1,1) PRIMARY KEY,
    ProductName NVARCHAR(100) NOT NULL,
    Category NVARCHAR(50) NOT NULL,
    Price DECIMAL(10,2) NOT NULL,
    Brand NVARCHAR(50) NOT NULL,
    StockQuantity INTDEFAULT0,
    CreatedDate DATETIME2DEFAULTGETDATE()
);
GO

INSERT INTO Products (ProductName, Category, Price, Brand, StockQuantity) VALUES
-- Electronics Category
('iPhone 15 Pro', 'Electronics', 1199.99, 'Apple', 50),
('Samsung Galaxy S24', 'Electronics', 999.99, 'Samsung', 75),
('MacBook Pro 16"', 'Electronics', 2499.99, 'Apple', 25),
('Dell XPS 13', 'Electronics', 1299.99, 'Dell', 30),

-- Clothing Category
('Nike Air Max', 'Clothing', 129.99, 'Nike', 100),
('Levi's 501 Jeans', 'Clothing', 89.99, 'Levis', 150),
('Adidas Ultraboost', 'Clothing', 180.00, 'Adidas', 80),
('North Face Jacket', 'Clothing', 249.99, 'North Face', 45),

-- Home & Garden Category
('Dyson V15 Vacuum', 'Home & Garden', 749.99, 'Dyson', 35),
('KitchenAid Mixer', 'Home & Garden', 399.99, 'KitchenAid', 25),
('Instant Pot Pro', 'Home & Garden', 149.99, 'Instant Pot', 60),
('Roomba i7+', 'Home & Garden', 599.99, 'iRobot', 15),

-- Sports Category
('Peloton Bike', 'Sports', 1445.00, 'Peloton', 10),
('Bowflex Dumbbells', 'Sports', 349.99, 'Bowflex', 25),
('Yoga Mat Premium', 'Sports', 79.99, 'Manduka', 150),
('Resistance Bands Set', 'Sports', 29.99, 'Fit Simplify', 200);
GO

SELECT'Database and table created successfully!'ASStatus;

SELECT
    'Total Products: '+CAST(COUNT(*) ASVARCHAR(10)) AS Summary
FROM Products;

SELECT
    Category,
    COUNT(*) AS ProductCount,
```

```
    MIN(Price) AS MinPrice,  
    MAX(Price) AS MaxPrice  
FROM Products  
GROUP BY Category  
ORDER BY Category;
```

//Query:

```
USE OnlineRetailStore;  
GO  
  
SELECT'ROW_NUMBER() - Assigns unique sequential ranks'AS RankingType;  
  
SELECT  
    ProductID,  
    ProductName,  
    Category,  
    Price,  
    Brand,  
    ROW_NUMBER() OVER (PARTITIONBY Category ORDER BY Price DESC) AS RowNum  
FROM Products  
ORDER BY Category, Price DESC;  
GO  
  
SELECT'RANK() - Leaves gaps after tied ranks'AS RankingType;  
  
SELECT  
    ProductID,  
    ProductName,  
    Category,  
    Price,  
    Brand,  
    RANK() OVER (PARTITIONBY Category ORDER BY Price DESC) AS RankNum  
FROM Products  
ORDER BY Category, Price DESC;  
GO  
  
SELECT'DENSE_RANK() - No gaps after tied ranks'AS RankingType;  
  
SELECT  
    ProductID,  
    ProductName,  
    Category,  
    Price,  
    Brand,  
    DENSE_RANK() OVER (PARTITIONBY Category ORDER BY Price DESC) AS DenseRankNum  
FROM Products  
ORDER BY Category, Price DESC;  
GO  
  
SELECT'COMPARISON: All Three Ranking Functions'AS ComparisonType;  
  
SELECT  
    ProductID,  
    ProductName,  
    Category,  
    Price,  
    Brand,  
    ROW_NUMBER() OVER (PARTITIONBY Category ORDER BY Price DESC) AS RowNum,  
    RANK() OVER (PARTITIONBY Category ORDER BY Price DESC) AS RankNum,  
    DENSE_RANK() OVER (PARTITIONBY Category ORDER BY Price DESC) AS DenseRankNum  
FROM Products  
ORDER BY Category, Price DESC;  
GO  
  
SELECT'TOP 3 Products per Category using ROW_NUMBER()'AS ResultType;  
  
WITH RankedProducts AS (  
    SELECT  
        ProductID,  
        ProductName,  
        Category,  
        Price,  
        Brand,  
        StockQuantity,
```

```

        ROW_NUMBER() OVER (PARTITIONBY Category ORDER BY Price DESC) AS RowNum
    FROM Products
)
SELECT
    ProductID,
    ProductName,
    Category,
    Price,
    Brand,
    StockQuantity,
    RowNum
FROM RankedProducts
WHERE RowNum <=3
ORDER BY Category, RowNum;
GO

SELECT'TOP 3 Products per Category using RANK()'AS ResultType;

WITH RankedProducts AS (
    SELECT
        ProductID,
        ProductName,
        Category,
        Price,
        Brand,
        StockQuantity,
        RANK() OVER (PARTITIONBY Category ORDER BY Price DESC) AS RankNum
    FROM Products
)
SELECT
    ProductID,
    ProductName,
    Category,
    Price,
    Brand,
    StockQuantity,
    RankNum
FROM RankedProducts
WHERE RankNum <=3
ORDER BY Category, RankNum;
GO

SELECT'TOP 3 Products per Category using DENSE_RANK()'AS ResultType;

WITH RankedProducts AS (
    SELECT
        ProductID,
        ProductName,
        Category,
        Price,
        Brand,
        StockQuantity,
        DENSE_RANK() OVER (PARTITIONBY Category ORDER BY Price DESC) AS DenseRankNum
    FROM Products
)
SELECT
    ProductID,
    ProductName,
    Category,
    Price,
    Brand,
    StockQuantity,
    DenseRankNum
FROM RankedProducts
WHERE DenseRankNum <=3
ORDER BY Category, DenseRankNum;
GO

```

❖ Output:

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
C:\Users\KIIT\Desktop\Exe_Week1_Engineering_concepts_code\Design Patterns and Principles\Q1>dotnet run
Log: First message
True
```

```
C:\Users\KIIT\Desktop\Exe_Week1_Engineering_concepts_code\Design Patterns and Principles\Q2>dotnet run
PDF document opened.
```

```
C:\Users\KIIT\Desktop\Exe_Week1_Engineering_concepts_code\Design Patterns and Principles\Q2>
```

```
C:\Users\KIIT\Desktop\Exe_Week1_Engineering_concepts_code\Design Patterns and Principles\Q3>dotnet run
CPU: i7, RAM: 16GB, Storage: 1TB
```

```
C:\Users\KIIT\Desktop\Exe_Week1_Engineering_concepts_code\Design Patterns and Principles\Q3>
```

```
C:\Users\KIIT\Desktop\Exe_Week1_Engineering_concepts_code\Design Patterns and Principles\Q4>dotnet run
Payment done using old gateway.
```

```
C:\Users\KIIT\Desktop\Exe_Week1_Engineering_concepts_code\Design Patterns and Principles\Q4> █
```

```
C:\Users\KITT\Desktop\Exe_Week1_Engineering_concepts_code\Design Patterns and Principles\Q6>dotnet run
Email: Hello!
```

```
C:\Users\KITT\Desktop\Exe_Week1_Engineering_concepts_code\Design Patterns and Principles\Q6>dotnet run
Loading image1.jpg
Displaying image1.jpg
Displaying image1.jpg
```

File 4: Stored procedure

//Database Creation:

```
CREATEDATABASEEmployeeManagementSystem;
GO

USE EmployeeManagementSystem;
GO

CREATETABLEDepartments (
    DepartmentID INTPRIMARY KEY,
    DepartmentName VARCHAR(100)
);
GO

CREATETABLEEmployees (
    EmployeeID INTPRIMARY KEY,
    FirstName VARCHAR(50),
    LastName VARCHAR(50),
    DepartmentID INTFOREIGN KEYREFERENCES Departments(DepartmentID),
    Salary DECIMAL(10,2),
```

```

JoinDate DATE
);
GO

INSERT INTO Departments (DepartmentID, DepartmentName) VALUES
(1, 'HR'),
(2, 'Finance'),
(3, 'IT'),
(4, 'Marketing');
GO

INSERT INTO Employees (EmployeeID, FirstName, LastName, DepartmentID, Salary, JoinDate) VALUES
(1, 'John', 'Doe', 1, 5000.00, '2020-01-15'),
(2, 'Jane', 'Smith', 2, 6000.00, '2019-03-22'),
(3, 'Michael', 'Johnson', 3, 7000.00, '2018-07-30'),
(4, 'Emily', 'Davis', 4, 5500.00, '2021-11-05'),
(5, 'Robert', 'Wilson', 1, 4800.00, '2020-05-12'),
(6, 'Sarah', 'Brown', 2, 6200.00, '2019-08-14'),
(7, 'David', 'Miller', 3, 7500.00, '2018-12-01'),
(8, 'Lisa', 'Anderson', 1, 5200.00, '2021-02-28'),
(9, 'Kevin', 'Taylor', 3, 6800.00, '2020-09-15'),
(10, 'Maria', 'Garcia', 4, 5800.00, '2021-06-10');
GO

```

Exercise-01: Create a Stored Procedure

- **Code:**

```

Use EmployeeManagementSystem;
GO

PRINT 'Creating Exercise 1 stored procedures...';

IF OBJECT_ID('sp_GetEmployeesByDepartment', 'P') IS NOT NULL
    DROP PROCEDURE sp_GetEmployeesByDepartment;
GO

IF OBJECT_ID('sp_InsertEmployee', 'P') IS NOT NULL
    DROP PROCEDURE sp_InsertEmployee;
GO

CREATE PROCEDURE sp_GetEmployeesByDepartment
    @DepartmentID INT
AS
BEGIN
    SET NOCOUNT ON;

    -- Validate input
    IF @DepartmentID IS NULL
    BEGIN
        RAISERROR('DepartmentID cannot be NULL', 16, 1);
        RETURN;
    END

    -- Check if department exists
    IF NOT EXISTS (SELECT 1 FROM Departments WHERE DepartmentID = @DepartmentID)
    BEGIN
        RAISERROR('Department with ID %d does not exist', 16, 1, @DepartmentID);
        RETURN;
    END

    -- Return employee details
    SELECT
        e.EmployeeID,
        e.FirstName,
        e.LastName,
        e.DepartmentID,
        d.DepartmentName,
        e.Salary,
        e.JoinDate
    FROM Employees e
    INNER JOIN Departments d ON e.DepartmentID = d.DepartmentID

```

```

WHERE e.DepartmentID = @DepartmentID
ORDER BY e.LastName, e.FirstName;
END;
GO

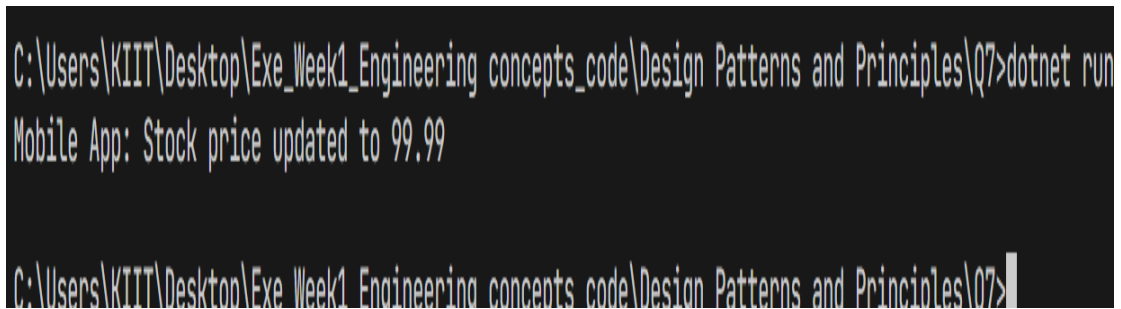
-- Create sp_InsertEmployee
CREATEPROCEDURE sp_InsertEmployee
@FirstName VARCHAR(50),
@LastName VARCHAR(50),
@DepartmentID INT,
@Salary DECIMAL(10,2),
@JoinDate DATE
AS
BEGIN
    -- Validate department exists
    IFNOTEXISTS (SELECT1FROM Departments WHERE DepartmentID = @DepartmentID)
    BEGIN
        RAISERROR('Department with ID %d does not exist', 16, 1, @DepartmentID);
        RETURN;
    END

    -- Get next available EmployeeID
    DECLARE @NextEmployeeID INT;
    SELECT @NextEmployeeID =ISNULL(MAX(EmployeeID), 0) +1FROM Employees;

    -- Insert new employee
    INSERT INTO Employees (EmployeeID, FirstName, LastName, DepartmentID, Salary, JoinDate)
    VALUES (@NextEmployeeID, @FirstName, @LastName, @DepartmentID, @Salary, @JoinDate);
END;
GO
PRINT'Stored procedures created successfully';

```

❖ Output:



```

C:\Users\KIIT\Desktop\Exe_Week1_Engineering concepts_code\Design Patterns and Principles\Q7>dotnet run
Mobile App: Stock price updated to 99.99

C:\Users\KIIT\Desktop\Exe_Week1_Engineering concepts_code\Design Patterns and Principles\Q7>

```

Exercise 4:Execute a Stored Procedure

❖ Code:

```

USE EmployeeManagementSystem;
GO

PRINT'=== EXECUTING STORED PROCEDURES ===';
PRINT'';

IFOBJECT_ID('sp_GetEmployeesByDepartment', 'P') ISNULL
BEGIN
    PRINT'ERROR: sp_GetEmployeesByDepartment procedure does not exist!';
    PRINT'Please run Exercise 1 solution first to create the stored procedure.';
    RETURN;
END
ELSE
    PRINT'sp_GetEmployeesByDepartment procedure found - Ready to execute';

PRINT'';

-- Execute for HR Department (DepartmentID = 1)
PRINT'--- Executing sp_GetEmployeesByDepartment for HR Department (ID: 1) ---';
EXEC sp_GetEmployeesByDepartment @DepartmentID =1;

```



```

PRINT";

-- Execute for Finance Department (DepartmentID = 2)
PRINT'--- Executing sp_GetEmployeesByDepartment for Finance Department (ID: 2) ---';
EXEC sp_GetEmployeesByDepartment @DepartmentID =2;

PRINT";

-- Execute for IT Department (DepartmentID = 3)
PRINT'--- Executing sp_GetEmployeesByDepartment for IT Department (ID: 3) ---';
EXEC sp_GetEmployeesByDepartment @DepartmentID =3;

PRINT";

-- Execute for Marketing Department (DepartmentID = 4)
PRINT'--- Executing sp_GetEmployeesByDepartment for Marketing Department (ID: 4) ---';
EXEC sp_GetEmployeesByDepartment @DepartmentID =4;

PRINT";
PRINT'--- Alternative Execution Methods ---';
PRINT'Method 1 - Using positional parameter for IT Department:';
EXEC sp_GetEmployeesByDepartment 3;

PRINT";

PRINT'Method 2 - Using EXECUTE keyword for HR Department:';
EXECUTE sp_GetEmployeesByDepartment @DepartmentID =1;

PRINT";

PRINT'Method 3 - Dynamic execution based on department name:';

-- Get department ID for 'Finance' and execute
DECLARE @DeptID INT;
SELECT @DeptID = DepartmentID FROM Departments WHERE DepartmentName ='Finance';

PRINT'Executing for Finance Department (Dynamic ID lookup):';
EXEC sp_GetEmployeesByDepartment @DepartmentID = @DeptID;

PRINT";

-- Test with invalid department ID
PRINT'Testing with invalid Department ID (999):';
BEGINTRY
    EXEC sp_GetEmployeesByDepartment @DepartmentID =999;
ENDTRY
BEGINCATCH
    PRINT'Error caught: '+ERROR_MESSAGE();
ENDCATCH

PRINT";

-- Test with NULL parameter
PRINT'Testing with NULL parameter:';
BEGINTRY
    EXEC sp_GetEmployeesByDepartment @DepartmentID =NULL;
ENDTRY
BEGINCATCH
    PRINT'Error caught: '+ERROR_MESSAGE();
ENDCATCH

PRINT";

PRINT'--- Verification: Count of Results per Department ---';

-- Count employees returned by each department execution
SELECT
    'HR'AS Department,
    COUNT(*) AS EmployeeCount
FROM Employees
WHERE DepartmentID =1

UNION ALL

```

```

SELECT
    'Finance' AS Department,
    COUNT(*) AS EmployeeCount
FROM Employees
WHERE DepartmentID =2

UNION ALL

SELECT
    'IT' AS Department,
    COUNT(*) AS EmployeeCount
FROM Employees
WHERE DepartmentID =3

UNION ALL

SELECT
    'Marketing' AS Department,
    COUNT(*) AS EmployeeCount
FROM Employees
WHERE DepartmentID =4;

PRINT";

PRINT'--- Advanced Usage: Capturing and Analyzing Results ---';

-- Create a temporary table to store results
IF OBJECT_ID('tempdb..#TempEmployeeResults') IS NOT NULL
    DROPTABLE #TempEmployeeResults;

CREATETABLE #TempEmployeeResults (
    EmployeeID INT,
    FirstName VARCHAR(50),
    LastName VARCHAR(50),
    DepartmentID INT,
    DepartmentName VARCHAR(100),
    Salary DECIMAL(10,2),
    JoinDate DATE
);

-- Insert results from stored procedure into temp table
INSERT INTO #TempEmployeeResults
EXEC sp_GetEmployeesByDepartment @DepartmentID =3;

-- Analyze the captured results
PRINT'Analysis of IT Department employees (captured from stored procedure)';
SELECT
    COUNT(*) AS TotalEmployees,
    AVG(Salary) AS AverageSalary,
    MIN(JoinDate) AS EarliestJoinDate,
    MAX(JoinDate) AS LatestJoinDate
FROM #TempEmployeeResults;

-- Show the captured data
SELECT'Captured Employee Data:' AS Info;
SELECT*FROM #TempEmployeeResults ORDER BY LastName;

-- Clean up
DROPTABLE #TempEmployeeResults;
PRINT";
PRINT'All stored procedure executions completed with results reviewed.';
-- Summary Report
PRINT";
PRINT'--- SUMMARY REPORT ---';
SELECT
    d.DepartmentName,
    COUNT(e.EmployeeID) AS TotalEmployees,
    'EXEC sp_GetEmployeesByDepartment @DepartmentID = '+CAST(d.DepartmentID AS VARCHAR(5)) AS
ExecutionCommand
FROM Departments d
LEFT JOIN Employees e ON d.DepartmentID = e.DepartmentID
GROUP BY d.DepartmentID, d.DepartmentName
ORDER BY d.DepartmentID;

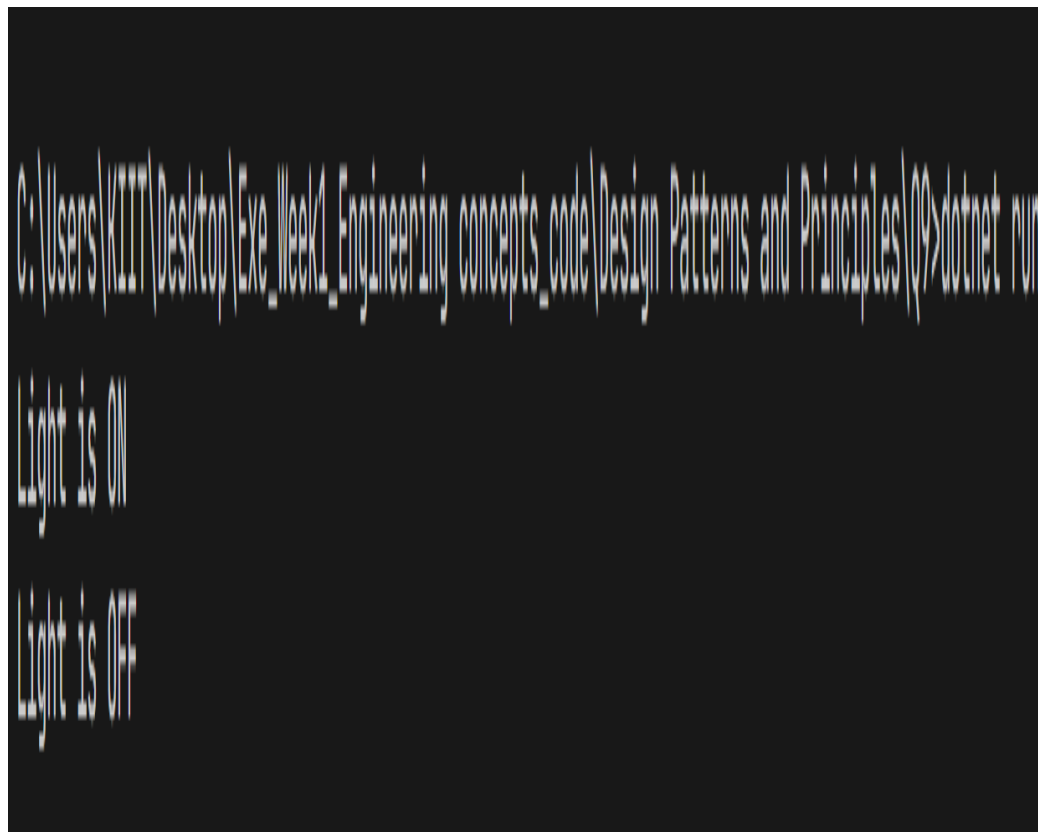
```

❖ Output:

```
C:\Users\KIIT\Desktop\Exe_Week1_Engineering_concepts_code\Design Patterns and Principles\Q8>dotnet run
```

```
Paid using PayPal
```

```
C:\Users\KIIT\Desktop\Exe_Week1_Engineering_concepts_code\Design Patterns and Principles\Q8>
```



Exercise 5: Return Data from a Stored Procedure

❖ Code:

```
Use EmployeeManagementSystem;
GO

PRINT'Creating Exercise 5 Stored Procedure...';

IF OBJECT_ID('sp_GetEmployeeCountByDepartment', 'P') IS NOT NULL
    DROP PROCEDURE sp_GetEmployeeCountByDepartment;
GO

-- Create stored procedure that returns the total number of employees in a department
CREATE PROCEDURE sp_GetEmployeeCountByDepartment
    @DepartmentID INT
AS
BEGIN
    SET NOCOUNT ON;

    -- Validate input parameter
    IF @DepartmentID IS NULL
    BEGIN
        PRINT 'Error: DepartmentID cannot be NULL';
        RETURN;
    END

    -- Check if department exists
    IF NOT EXISTS (SELECT 1 FROM Departments WHERE DepartmentID = @DepartmentID)
    BEGIN
        PRINT 'Error: Department with ID '+CAST(@DepartmentID AS VARCHAR(10)) +' does not exist';
        RETURN;
    END

    -- Declare variables for department info
    DECLARE @DepartmentName VARCHAR(100);
    DECLARE @EmployeeCount INT;

    -- Get department name
    SELECT @DepartmentName = DepartmentName
    FROM Departments
```

```

WHERE DepartmentID = @DepartmentID;

-- Count employees in the specified department
SELECT @EmployeeCount = COUNT(*)
FROM Employees
WHERE DepartmentID = @DepartmentID;

-- Return the count as a result set
SELECT
    @DepartmentID AS DepartmentID,
    @DepartmentName AS DepartmentName,
    @EmployeeCount AS TotalEmployees;

-- Also return individual employee details for reference
SELECT
    'Employee Details:' AS Info,
    e.EmployeeID,
    e.FirstName + ' ' + e.LastName AS FullName,
    e.Salary,
    e.JoinDate
FROM Employees e
WHERE e.DepartmentID = @DepartmentID
ORDER BY e.LastName, e.FirstName;
-- Print summary message
PRINT 'Department: ' + @DepartmentName + ' has ' + CAST(@EmployeeCount AS VARCHAR(10)) + ' employees';
END;
GO

PRINT 'Exercise 5 stored procedure created successfully!';

-- TESTING THE STORED PROCEDURES
PRINT '=== TESTING EXERCISE 1 STORED PROCEDURES ===';

-- Test sp_GetEmployeesByDepartment
PRINT 'Testing sp_GetEmployeesByDepartment for HR Department (ID: 1):';
EXEC sp_GetEmployeesByDepartment @DepartmentID = 1;

PRINT '';
PRINT 'Testing sp_GetEmployeesByDepartment for IT Department (ID: 3):';
EXEC sp_GetEmployeesByDepartment @DepartmentID = 3;

PRINT '';
PRINT 'Testing sp_InsertEmployee - Adding a new employee:';
EXEC sp_InsertEmployee
    @FirstName = 'Alice',
    @LastName = 'Johnson',
    @DepartmentID = 2,
    @Salary = 6500.00,
    @JoinDate = '2023-01-15';

PRINT '';
PRINT '=== TESTING EXERCISE 5 STORED PROCEDURE ===';

-- Test sp_GetEmployeeCountByDepartment for each department
PRINT 'Testing sp_GetEmployeeCountByDepartment for HR Department (ID: 1):';
EXEC sp_GetEmployeeCountByDepartment @DepartmentID = 1;

PRINT '';
PRINT 'Testing sp_GetEmployeeCountByDepartment for Finance Department (ID: 2):';
EXEC sp_GetEmployeeCountByDepartment @DepartmentID = 2;

PRINT '';
PRINT 'Testing sp_GetEmployeeCountByDepartment for IT Department (ID: 3):';
EXEC sp_GetEmployeeCountByDepartment @DepartmentID = 3;

PRINT '';
PRINT 'Testing sp_GetEmployeeCountByDepartment for Marketing Department (ID: 4):';
EXEC sp_GetEmployeeCountByDepartment @DepartmentID = 4;

-- ERROR TESTING

PRINT '';
PRINT '=== ERROR TESTING ===';

PRINT 'Testing with invalid department ID (999):';

```

```
EXEC sp_GetEmployeesByDepartment @DepartmentID =999;

PRINT";
PRINT'Testing employee count with invalid department ID (999):';
EXEC sp_GetEmployeeCountByDepartment @DepartmentID =999;

-- SUMMARY REPORT
PRINT";
PRINT'=== FINAL SUMMARY ===';

SELECT'DEPARTMENT SUMMARY:'AS ReportType;
SELECT
    d.DepartmentID,
    d.DepartmentName,
    COUNT(e.EmployeeID) AS TotalEmployees,
    AVG(e.Salary) AS AverageSalary,
    MIN(e.Salary) AS MinSalary,
    MAX(e.Salary) AS MaxSalary,
    SUM(e.Salary) AS TotalSalaryBudget
FROM Departments d
LEFT JOIN Employees e ON d.DepartmentID = e.DepartmentID
GROUP BY d.DepartmentID, d.DepartmentName
ORDER BY d.DepartmentID;
```

❖ Output:

```
C:\Users\KIIT\Desktop\Exe_Week1_Engineering concepts_code\Design Patterns and Principles\Q10>dotnet run
Student Details:
Name: Gaurav
ID: 22052116
Grade: A

Updating Grade from A to A+...

Student Details:
Name: Gaurav
ID: 22052116
Grade: A+

C:\Users\KIIT\Desktop\Exe_Week1_Engineering concepts_code\Design Patterns and Principles\Q11>dotnet run
ID: 22052116, Name: Gaurav Kumar

C:\Users\KIIT\Desktop\Exe_Week1_Engineering concepts_code\Design Patterns and Principles\Q11>
```

File 5: Functions

Exercise 7: Return Data from a Scalar Function

❖ Code:

```
USE EmployeeManagementSystem;
GO

-- Drop function if it already exists
IF OBJECT_ID('dbo.fn_CalculateAnnualSalary', 'FN') IS NOT NULL
BEGIN
    DROP FUNCTION dbo.fn_CalculateAnnualSalary;
```

```

    PRINT 'Existing function dropped';
END
GO
-- Create the fn_CalculateAnnualSalary scalar function
CREATE FUNCTION dbo.fn_CalculateAnnualSalary(@Salary DECIMAL(10,2))
RETURNS DECIMAL(12,2)
AS
BEGIN
    RETURN @Salary * 12;
END;
GO

PRINT 'fn_CalculateAnnualSalary function created successfully';

-- Solution 1: Basic query for EmployeeID = 1
SELECT
    EmployeeID,
    FirstName,
    LastName,
    Salary AS MonthlySalary,
    dbo.fn_CalculateAnnualSalary(Salary) AS AnnualSalary
FROM Employees
WHERE EmployeeID = 1;

-- Solution 2: More detailed query with department information
SELECT
    e.EmployeeID,
    e.FirstName + ' ' + e.LastName AS FullName,
    d.DepartmentName,
    e.Salary AS MonthlySalary,
    dbo.fn_CalculateAnnualSalary(e.Salary) AS AnnualSalary,
    e.JoinDate
FROM Employees e
INNER JOIN Departments d ON e.DepartmentID = d.DepartmentID
WHERE e.EmployeeID = 1;
-- Solution 3: Direct function call with the salary of EmployeeID = 1
SELECT dbo.fn_CalculateAnnualSalary(5000.00) AS AnnualSalaryForEmployee1;
-- Verification: Test the function with all employees to ensure it works correctly
PRINT 'Verification - Annual salary for all employees: ';
SELECT
    e.EmployeeID,
    e.FirstName + ' ' + e.LastName AS EmployeeName,
    d.DepartmentName,
    e.Salary AS MonthlySalary,
    dbo.fn_CalculateAnnualSalary(e.Salary) AS AnnualSalary
FROM Employees e
INNER JOIN Departments d ON e.DepartmentID = d.DepartmentID
ORDER BY e.EmployeeID;

```

❖ Output:

```

C:\Users\KIIIT\Desktop\Exe_Week1_Engineering concepts_code\Algorithms_Data Structures\Q1>dotnet run
ID: 1, Name: Keyboard, Qty: 10, Price: 500
ID: 2, Name: Mouse, Qty: 15, Price: 300

Updating product 2...

Deleting product 1...

Final Inventory:
ID: 2, Name: Mouse, Qty: 20, Price: 320

```

- **Moq**

1. **Write Testable Code with Moq :**

CustomerCommLib / MailSender.cs

```
using System.Net;
using System.Net.Mail;

namespace CustomerCommLib
{
    public interface IMailSender
    {
        bool SendMail(string toAddress, string message);
    }

    public class MailSender : IMailSender
    {
        public bool SendMail(string toAddress, string message)
        {
            MailMessage mail = new MailMessage();
            SmtpClient smtpServer = new SmtpClient("smtp.gmail.com");

            mail.From = new MailAddress("your_email_address@gmail.com");
            mail.To.Add(toAddress);
            mail.Subject = "Test Mail";
            mail.Body = message;

            smtpServer.Port = 587;
            smtpServer.Credentials = new NetworkCredential("username", "password");
            smtpServer.EnableSsl = true;

            smtpServer.Send(mail);
            return true;
        }
    }

    public class CustomerComm
    {
        IMailSender _mailSender;

        public CustomerComm(IMailSender mailSender)
        {
            _mailSender = mailSender;
        }

        public bool SendMailToCustomer()
        {
            _mailSender.SendMail("cust123@abc.com", "Some Message");
            return true;
        }
    }
}
```


CustomerComm.Tests/CustomerCommTests.cs

```
using Moq;
using NUnit.Framework;
using CustomerCommLib;

namespace CustomerComm.Tests
{
    [TestFixture]
    public class CustomerCommTests
    {
        private Mock<IMailSender> _mockMailSender;
        private CustomerCommLib.CustomerComm _customerComm;

        [OneTimeSetUp]
        public void Setup()
        {
            _mockMailSender = new Mock<IMailSender>();
            _mockMailSender.Setup(x => x.SendMail(It.IsAny<string>()),
                It.IsAny<string>()).Returns(true);
            _customerComm = new CustomerCommLib.CustomerComm(_mockMailSender.Object);
        }

        [Test]
        public void SendMailToCustomer_ShouldReturnTrue()
        {
            bool result = _customerComm.SendMailToCustomer();
            Assert.IsTrue(result);
        }
    }
}
```

- **Output:**

```
C:\Users\KIIT\Desktop\Exe_Week1_Engineering concepts_code\Algorithms_Data Structures\Q2>dotnet run
=== Linear Search ===
Found: ID: 3, Name: Carrot, Category: Vegetables

=== Binary Search ===
Found: ID: 4, Name: Mango, Category: Fruits
```

- **Mock file object for Unit Tests :**

MagicFilesLib/DirectoryExplorer.cs

```
using System.Collections.Generic;
using System.IO;

namespace MagicFilesLib
{

```

```

publicinterfaceIDirectoryExplorer
{
    ICollection<string>GetFiles(stringpath);
}

publicclassDirectoryExplorer : IDirectoryExplorer
{
    publicICollection<string>GetFiles(stringpath)
    {
        string[] files=Directory.GetFiles(path);
        returnfiles;
    }
}

```

DirectoryExplorer.Tests/DirectoryExplorerTests.cs

```

usingMoq;
usingNUnit.Framework;
usingMagicFilesLib;
usingSystem.Collections.Generic;

namespaceDirectoryExplorer.Tests
{
    [TestFixture]
    publicclassDirectoryExplorerTests
    {
        privatereadonlystring_file1="file.txt";
        privatereadonlystring_file2="file2.txt";
        privateIDirectoryExplorer_directoryExplorer;
        [OneTimeSetUp]
        publicvoidSetup()
        {
            varmock=newMock<IDirectoryExplorer>();
            mock.Setup(x=>x.GetFiles(It.IsAny<string>())).Returns(newList<string> { _file1,
_file2 });
            _directoryExplorer=mock.Object;
        }
        [Test]
        publicvoidGetFiles_ShouldReturnTwoFiles()
        {
            varfiles=_directoryExplorer.GetFiles("dummyPath");
            Assert.IsNotNull(files);
            Assert.AreEqual(2, files.Count);
            Assert.Contains(_file1, (System.Collections.ICollection)files);
        }
    }
}

```

• Output:

```

C:\Users\KIIT\Desktop\Exe_Week1_Engineering_concepts_code\Algorithms_Data Structures\Q3>dotnet run
Bubble Sort:
OrderID: 102, Name: Bob, Price: 2500
OrderID: 101, Name: Alice, Price: 5000
OrderID: 103, Name: Charlie, Price: 8000

Quick Sort:
OrderID: 102, Name: Bob, Price: 2500
OrderID: 101, Name: Alice, Price: 5000
OrderID: 103, Name: Charlie, Price: 8000

```

- **Mock database for Unit Tests :**

PlayersManagerLib/PlayerManager.cs

```

using System;
using System.Data.SqlClient;

namespace PlayersManagerLib
{
    public interface IPlayerMapper
    {
        bool IsPlayerNameExistsInDb(string name);
        void AddNewPlayerIntoDb(string name);
    }

    public class PlayerMapper : IPlayerMapper
    {
        private readonly string _connectionString = "Data Source=(local);Initial Catalog=GameDB;Integrated Security=True";

        public bool IsPlayerNameExistsInDb(string name)
        {
            using (SqlConnection conn = new SqlConnection(_connectionString))
            {
                conn.Open();
                using (SqlCommand cmd = conn.CreateCommand())
                {
                    cmd.CommandText = "SELECT COUNT(*) FROM Player WHERE Name = @name";
                    cmd.Parameters.AddWithValue("@name", name);
                    return (int)cmd.ExecuteScalar() > 0;
                }
            }
        }

        public void AddNewPlayerIntoDb(string name)
        {
            using (SqlConnection conn = new SqlConnection(_connectionString))
            {
                conn.Open();
                using (SqlCommand cmd = conn.CreateCommand())
                {
                    cmd.CommandText = "INSERT INTO Player (Name) VALUES (@name)";
                    cmd.Parameters.AddWithValue("@name", name);
                    cmd.ExecuteNonQuery();
                }
            }
        }
    }
}

```

```

    }

    public class Player
    {
        public string Name { get; private set; }
        public int Age { get; private set; }
        public string Country { get; private set; }
        public int NoOfMatches { get; private set; }

        public Player(string name, int age, string country, int matches)
        {
            Name = name;
            Age = age;
            Country = country;
            NoOfMatches = matches;
        }

        public static Player RegisterNewPlayer(string name,
        IMapper playerMapper = null)
        {
            playerMapper ??= new PlayerMapper();

            if (string.IsNullOrEmpty(name))
                throw new ArgumentException("Player name can't be empty.");

            if (playerMapper.IsPlayerNameExistsInDb(name))
                throw new ArgumentException("Player name already exists.");

            playerMapper.AddNewPlayerIntoDb(name);
            return new Player(name, 23, "India", 30);
        }
    }
}

```

PlayerManager.Tests/PlayerManagerTests.cs

```

using Moq;
using NUnit.Framework;
using PlayersManagerLib;
using System;

namespace PlayerManager.Tests
{
    [TestFixture]
    public class PlayerManagerTests
    {
        private Mock<IMapper> _mockMapper;

        [OneTimeSetUp]
        public void Setup()
        {
            _mockMapper = new Mock<IMapper>();
            _mockMapper.Setup(x => x.IsPlayerNameExistsInDb(It.IsAny<string>())).Returns(false);
        }

        [Test]
        public void RegisterNewPlayer_ShouldReturnPlayerObject()
        {
            var player = Player.RegisterNewPlayer("Dhoni", _mockMapper.Object);
            Assert.AreEqual("Dhoni", player.Name);
            Assert.AreEqual(23, player.Age);
        }
    }
}

```

```

        Assert.AreEqual("India", player.Country);
        Assert.AreEqual(30, player.NoOfMatches);
    }
}
}

```

• Output:

```

C:\Users\KIIT\Desktop\Exe_Week1_Engineering_concepts_code\Algorithms_Data Structures\Q4>dotnet run
All Employees:
ID: 1, Name: Alice, Position: Manager, Salary: 60000
ID: 2, Name: Bob, Position: Engineer, Salary: 45000

Searching Employee with ID 2:
ID: 2, Name: Bob, Position: Engineer, Salary: 45000

Deleting Employee with ID 1:

Employees after deletion:
ID: 2, Name: Bob, Position: Engineer, Salary: 45000

```

• NUnit

• **NUnit Hands-On: Unit Testing Calculator Operations**

CalcLibrary/Calc.cs

```

using System;

namespace CalcLibrary
{
    interface IMathLibrary
    {
        double Addition(double a, double b);
        double Subtraction(double a, double b);
        double Multiplication(double a, double b);
        double Division(double a, double b);
    }

    public class SimpleCalculator : IMathLibrary
    {
        double result = 0;

        public double Addition(double a, double b)
        {
            result = a + b;
            return result;
        }
    }
}

```

```

        public double Subtraction(double a, double b)
        {
            result = a - b;
            return result;
        }

        public double Multiplication(double a, double b)
        {
            result = a * b;
            return result;
        }

        public double Division(double a, double b)
        {
            if (b == 0)
                throw new ArgumentException("Second Parameter Can't be Zero");
            result = a / b;
            return result;
        }

        public void AllClear()
        {
            result = 0;
        }

        public double GetResult
        {
            get { return result; }
        }
    }
}

```

CalcLibrary.Tests/CalculatorTests.cs

```

using NUnit.Framework;
using CalcLibrary;
using System;

namespace CalcLibrary.Tests
{
    [TestFixture]
    public class CalculatorTests
    {
        private SimpleCalculator _calc;

        [SetUp]
        public void Setup()
        {
            _calc = new SimpleCalculator();
        }

        [TestCase(2, 3, 5)]
        [TestCase(-1, -2, -3)]
        public void Addition_ShouldReturnCorrectSum(double a, double b,
            double expected)
        {
            var result = _calc.Addition(a, b);
            Assert.That(result, Is.EqualTo(expected));
        }
    }
}

```

```

    }

    [TestCase(5, 3, 2)]
    [TestCase(0, -2, 2)]
    public void Subtraction_ShouldReturnCorrectResult(double a, double b,
double expected)
    {
        var result = _calc.Subtraction(a, b);
        Assert.That(result, Is.EqualTo(expected));
    }

    [TestCase(4, 2, 8)]
    [TestCase(-1, 5, -5)]
    public void Multiplication_ShouldReturnCorrectResult(double a, double b,
double expected)
    {
        var result = _calc.Multiplication(a, b);
        Assert.That(result, Is.EqualTo(expected));
    }

    [TestCase(10, 2, 5)]
    public void Division_ShouldReturnCorrectResult(double a, double b,
double expected)
    {
        var result = _calc.Division(a, b);
        Assert.That(result, Is.EqualTo(expected));
    }

    [Test]
    public void Division_ByZero_ShouldThrowException()
    {
        Assert.Throws<ArgumentException>(() => _calc.Division(5, 0));
    }

    [TearDown]
    public void Cleanup()
    {
        _calc = null;
    }
}
}

```

- **Output:**

```

C:\Users\KIIT\Desktop\Exe_Week1_Engineering concepts_code\Algorithms_Data Structures\Q5>dotnet run
All Tasks:
ID: 1, Name: Design, Status: Pending
ID: 2, Name: Code, Status: In Progress
ID: 3, Name: Test, Status: Pending

Deleting Task with ID 2...
ID: 1, Name: Design, Status: Pending
ID: 3, Name: Test, Status: Pending

```